

课程介绍：《编程训练 108 题》

作者：C3 程序猿

课程总结常见问题进行讲解：

- 1、爱心效果类(5 道题目)
- 2、图形输出效果类(9 道题目)
- 3、整数操作类(10 道题目)
- 4、数列计算类(6 道题目)
- 5、递归应用类(8 道题目)
- 6、数组与指针类(10 道题目)
- 7、随机数生成类(9 道题目)
- 8、日历显示类(7 道题目)
- 9、字符串操作类(18 道题目)
- 10、二进制操作类(11 道题目)
- 11、文件操作类(5 道题目)
- 12、逻辑思维题(11 道题目)

课程学习建议：

课程是针对 C 语言基础语法，学完 C 语言基础就可以看这个课，可以有效练习语法应用，锻炼基础的编程思维，为后续的学习打下坚实的动手基础。

题目中部分题目比较难，可以战略性放弃，我们目的就是练手，不是练脾气，明明不会还在那儿卡着，闲得慌。

108 道题目根据自己的情况学习，不必按顺序学习，也不是必须全部学习，可以跟着学 50 道，感觉学的挺顺利，挺开心，挺有收货，那就再学 50 个，多多益善。有的小伙子着急考试，时间紧迫，就少练几个，每个类型练 3 个 2 个也行。

六部分内容：

1、硬输出，用 printf 自己拼一个爱心：可以做的很漂亮

[illegible]

注意 1: 要大一点儿, 不然心型会很干瘪, 越大越饱满。

[illegible]

在网上搜索的代码，有一些是有很多的 for 循环，目的是将心拆分，每个有规律的部分用循环写。所以最后一堆 for 循环。不同的人，循环规律也不一样，所以循环的设计也有差别。

也可以将每一行的空格数装入数组，这样规律就统一了，如下

```
void Xin_4(void)
{
    int gui[15] = { 7, 4, 2, 1, 0, 0, 1, 2, 4, 6, 8, 11, 14, 17, 19 }; //每一行首的空格数
    int zhongkong[3] = { 9, 3, 1 }; //前三行中间的空格数
    int i, j, lie = 41; //41 列
    //前三行
    for (i = 0; i < 3; i++)
    {
        for (j = 0; j < gui[i]; j++) //前三行首空格
            putchar(' ');

        int n = (lie - zhongkong[i] - 2 * gui[i]) / 2; //星号数
        for (j = 0; j < n; j++)
            putchar('*');

        for (j = 0; j < zhongkong[i]; j++) //中间空格
            putchar(' ');

        for (j = 0; j < n; j++) //第二列星号
            putchar('*');

        putchar('\n');
    }
    //后 12 行
    for (i = 3; i < 15; i++)
    {
        for (j = 0; j < gui[i]; j++) //行首空格
            putchar(' ');

        for (j = 0; j < lie - 2 * gui[i]; j++) //星号 不间断的
            putchar('*');

        putchar('\n');
    }
}
```

网上一段很干练的代码，这不是我写的，我也不知道原作者是谁，很帅气的代码。
这种代码大家会有疑问是怎么写出来的，我也不知道。
详细分析代码的执行结果，从而得到代码的执行过程，就可以了。

```
void Xin_5(void)
{
    for (float y = 1.5f; y > -1.5f; y -= 0.1f)
    {
        for (float x = -1.5f; x < 1.5f; x += 0.05f)
        {
            float a = x * x + y * y - 1;
            float n = a * a * a - x * x * y * y * y; //我是 不知道作者怎么想出来的
            putchar(n <= 0.0f ? '*' : ' '); //当 n 的值小于等于 0，输出*，否则输出空格
        }
        putchar('\n');
    }
}
```

$a = -1.5 * -1.5 + 1.5 * 1.5 - 1 = 3.5$

$n = 3.5 * 3.5 * 3.5 - -1.5 * -1.5 * 1.5 * 1.5 * 1.5 = 42.875 - 7.59375 > 0$ 输出空格

爱心合拢效果

将爱心分成 4 个部分，中心点坐标是 7,20

0,0 到 7,20 这部分输出在控制台左上角起始 0,0 位置

0,21 到 7,41 这部分输出在控制台右上角起始 0,41 位置

8,0 到 15,20 这部分输出在控制台左下角起始 18,0 位置

8,21 到 15,41 这部分输出在控制台右下角起始 18,41 位置

参数 1,2 是指示控制台坐标 参数 3,4 是爱心的四块起始坐标

```
void dong_2(int hang, int lie, int strhang, int strlie)
```

```
{
    int i = 0, j = 0, m = hang, n = lie;
    for (i = 0; i <= 7; i++)
    {
        for (j = 0; j <= 20; j++)
        {
            if ('*' == str[strhang + i][strlie + j])
            {
                Goto(hang+i, lie+j);
                putchar('*');
            }
        }
    }
}
```

```
void dong_3(void)
```

```
{
    int n;
    for (n = 0; n <= 5; n++)
    {
        system("cls"); //清空控制台
        dong_2(n, 2*n, 0, 0);
        dong_2(18-n, 2 * n, 8, 0);

        dong_2(n, 41- 2 * n, 0, 21);
        dong_2(18-n, 41- 2 * n, 8, 21);
        Sleep(100);
    }
}
```

第一类：图形输出类

1、输出直角三角形，如下：

```
*
***
*****
*****
*****
```

分析：

(1) 1 3 5 7 9，每一行的星号个数为该行数的 $2*n - 1$ ， n 表示第 1, 2, 3, 4, 5 行的行号，当然每一行的星号数，由大家决定，规律自行查找

得到如下代码

```
int n = 1, m = 5;           //m 表示几行，可随意
int i = 0;
for (n = 1; n <= m; n++)    //输出 5 行星号, 5 可变
{
    for (i = 0; i < 2 * n - 1; i++) //输出每行的*
        putchar('*');
    putchar('\n');           //输出完一行输出换行
}
```

得到结果如下：

```
*
***
*****
*****
*****
```

封装函数： m 是可变量，控制输出的三角形的大小，可通过函数参数传递

```
void zhijiao(int m)
{
    int n = 1, i = 0;
    for (n = 1; n <= m; n++)    //输出 5 行星号, 5 可变
    {
        for (i = 0; i < 2 * n - 1; i++) //输出每行的*
            putchar('*');
        putchar('\n');           //输出完一行输出换行
    }
}
```

函数调用

```
zhijiao(6);
zhijiao(10);
zhijiao(11);
```


2、输出等腰三角形，如下：

```
 *
***
*****
*****
*****
```

对比题目 1 的直角三角形

```
 *
***
*****
*****
*****
```

想要变成等腰三角形，只需在星号前面输出空格，将星号顶到中间

```
 *
***
*****
*****
*****
```

分析：5 行开头需要空格数依次 4, 3, 2, 1, 0，是跟行数有数学关系，行数-行号
得到代码如下

```
int n = 1, m = 5;           //m 行数，n 行号，可随意
int i = 0, j = 0;
for (n = 1; n <= m; n++)    //输出 5 行星号, 5 可变
{
    for (j = m - n; j > 0; j--) //输出每行前的空格
        putchar(' ');
    for (i = 0; i < 2 * n - 1; i++) //输出每行的*
        putchar('*');
    putchar('\n');           //输出完一行输出换行
}
```

封装函数：m 是可变量，控制输出的三角形的大小，可通过函数参数传递

```
void dengyao(int m)
{
    int n = 1, i = 0, j = 0;
    for (n = 1; n <= m; n++) //输出 5 行星号, 5 可变
    {
        for (j = m - n; j > 0; j--) //输出每行前的空格
            putchar(' ');
        for (i = 0; i < 2 * n - 1; i++) //输出每行的*
            putchar('*');
        putchar('\n');           //输出完一行输出换行
    }
}
```

函数调用

```
dengyao(6);
dengyao(10);
dengyao(11);
```

3、输出菱形，如下：

```
*
***
*****
*****
***
*
```

分析：分成 2 部分，上半部分是正的等腰三角形，下半部分是倒的等腰三角形

上半部分代码同：dengyao(4);是一模一样的

按照该思路写个倒三角的代码：

1、下方三角前的空格数是 1, 2, 3,

2、下方三角的星号数是 5, 3, 1,

```
for (n = 1; n < m; n++)          //n 行数，< m 上半边<=5 输出 5 行，下边<5 输出 4 行
{
    for (j = 0; j < n; j++)        //输出每行前的空格 1, 2, 3, 4... 就可以利用 n
        putchar(' ');
    for (i = 0; i < 2*(m-n)-1; i++) //输出每行的*, ... 9, 7, 5, 3, 1, m 是总行数配合 n
        putchar('*');
    putchar('\n');                  //输出完一行输出换行
}
```

完整代码如下

```
void lingxing(int m)
{
    int n = 1, i = 0, j = 0;
    for (n = 1; n <= m; n++)        //输出 5 行星号, 5 可变
    {
        for (j = m - n; j > 0; j--)  //输出每行前的空格
            putchar(' ');
        for (i = 0; i < 2 * n - 1; i++) //输出每行的*
            putchar('*');
        putchar('\n');                //输出完一行输出换行
    }

    for (n = 1; n < m; n++)          //行数，< m 上半边输出 5 行，下边输出 4 行
    {
        for (j = 0; j < n; j++)        //输出每行前的空格 1, 2, 3...
            putchar(' ');
        for (i = 0; i < 2*(m-n)-1; i++) //输出每行的*
            putchar('*');
        putchar('\n');                //输出完一行输出换行
    }
}
```

4、输出 6 边形

```
***
*****
*****
*****
***
```

这个跟菱形的书写思路是一样的

图形规律：m 表示边长，比如 m=3，那么每一条边都是 3 个星号

总行数就是 $m+m-1 == 2*m-1 == 5$ 行

每一行的行号：-2 -1 0 1 2 行号 n, $n=1-m$ $n < m$ $n++$

```
int n , m = 3; //n 行号, 2*m-1 是总行数
for (n = 1 - m; n < m; n++)
{
    //总共循环次数是 2*m-1 次
}
```

每一行的空格数：2 1 0 1 2 每一行的空格数为行号的绝对值

```
int n , m = 3; //n 行号, 2*m-1 是总行数
int i;
for (n = 1 - m; n < m; n++)
{
    for (i = 0; i < abs(n); i++)//输出空格
        putchar(' ');
}
```

每一行的星号数：3 5 7 5 3 $n < 0$ ，星号增量为 2， $n \geq 0$ ，星号增量为 -2

```
int n , m = 3; //n 行号, m 是边长, 2*m-1 是总行数
int i, j;
int t = m; //定义中间变量使用 m
for (n = 1 - m; n < m; n++)
{
    //输出空格
    for (i = 0; i < abs(n); i++)
        putchar(' ');
    //输出星号
    for (j = 0; j < t; j++)
        putchar('*');
    //t 用来控制星号数量
    if (n < 0)
        t += 2;
    else
        t -= 2;
    putchar('\n');
}
```

26 字母构成直角三角形

A
BC
DEF
GHIJ
KLNMO
PQRSTU
VWXYZ

特点:

1: 行号, 1,2,3,4,5,6..... 总行数不确定, 由字母(26)个决定

```
int n = 1; //行号
while (1) //循环次数不确定时, 先用死循环
{
    //输出每一行
    n++; //一次循环输出一行
}
```

2: 每行字母数: 1,2,3,4,5,6..... 跟行号一样

```
int n = 1; //行号
int i = 0; //循环控制变量
while (1) //循环次数不确定时, 先用死循环
{
    for (i = 0; i < n; i++)
        putchar('*'); //输出字符, 先用*代替字母
    n++; //一次循环输出一行
    putchar('\n'); //输出换行
}
```

3: 字母是依次增加的, 所以可以用变量 c

```
int n = 1; //行号
int i = 0; //循环控制变量
char c = 'A'; //字符变量 c, 装字母
while (1) //循环次数不确定时, 先用死循环
{
    for (i = 0; i < n; i++)
    {
        putchar(c); //输出字符
        c++; //输出完自加 1, 输出一个自加 1
    }
    n++; //一次循环输出一行
    putchar('\n'); //输出换行
}
```

4: 循环次数控制, 由 26 个字母输出完了, 就结束循环

```
int n = 1; //行号
int i = 0; //循环控制变量
```

```

char c = 'A';
while (c <= 'Z')    //当 c>z 是结束
{
    for (i = 0; i < n && c <= 'Z'; i++)//当 c>z 是结束，因为最后一次不一定满行
    {
        putchar(c);    //输出字符
        c++;           //输出完自加 1, 输出一个自加 1
    }
    n++;               //一次循环输出一行
    putchar('\n');     //输出换行
}

```

总结：初学者由于经验问题，不能一眼看出所有细节，但是肯定能知道程序由哪些逻辑部分组成，耐心的依次完成每个部分。

不要纠结变量定义的多少，有的人就写一个变量，在那儿各种用，用来用去，用蒙圈了。每个部分需要变量了，你就写个 int a，也不累。

代码优化什么的，初学者也不用考虑，优化的能力是经验实践出的个人能力。初学者不要要求这个，能把想法写出来，这就足够了。另外，这么简单的功能，根本不涉及优化。

字母中间对称三角形，如下

```
A
ABA
ABCB
ABCD
ABCD
```

等腰三角形：行数 m ，传递。行号 n ，1,2,3,4,5.....逻辑跟前面的一样

空格数的规律： $m-1, m-2, m-3, m-4 \dots m-n$

每一行字母数规律： $1, 3, 5, 7, 9 \dots 2*n-1$

```
void c_zhijiao(int m)
{
    int n = 1, i, j;
    for (n = 1; n <= m; n++) //行遍历
    {
        for (i = 1; i <= m - n; i++) //输出空格
            putchar(' ');
        for (j = 1; j <= 2*n-1; j++) //输出字符
            putchar('*');
        putchar('\n'); //输出换行
    }
}
```

字母规律：每一行的字母是回文的，并且连续

中间对称字母跟行的规律：第 1 行 A，第 2 行 B，第 3 行 C，第 4 行 D.....

中间字母跟行的关系： $\text{char } c = 64 + n$ A 对应的 ASCII 码是 65

整行字母的变化规律是： $j = 1 - n$ 开始， $j < n$ ，比如第 4 行： $-3 \sim 3$

$j < 0$ 时 字符是 $c + j$ ----- $j \geq 0$ 是 字符是 $c - j$

```
void c_zhijiao(int m)
{
    int n = 1, i, j;
    for (n = 1; n <= m; n++) //行遍历
    {
        for (i = 1; i <= m - n; i++) //输出空格
            putchar(' ');
        //输出字符
        char c = 64 + n;
        for (j = 1 - n; j < n; j++)
        {
            if (j < 0) putchar(c + j);
            else putchar(c - j);
        }
        putchar('\n'); //输出换行
    }
}
```

封装成函数

```
void liubianxing(int m)
{
    int n;          //行号
    int i, j;
    int t = m;      //定义中间变量使用 m
    for (n = 1 - m; n < m; n++)
    {
        //输出空格
        for (i = 0; i < abs(n); i++)
            putchar(' ');
        //输出星号
        for (j = 0; j < t; j++)
            putchar('*');
        //t 用来控制星号数量
        if (n < 0)
            t += 2;
        else
            t -= 2;
        putchar('\n');
    }
}
```

杨辉三角形

```

      1
     1 1
    1 2 1
   1 3 3 1
  1 4 6 4 1
 1 5 10 10 5 1
1 6 15 20 15 6 1
1 7 21 35 35 21 7 1
1 8 28 56 70 56 28 8 1
1 9 36 84 126 126 84 36 9 1
```

规则：

行列：n 行 n 列，n 为 1,2,3,4,5,6,7,8.....连续的

数规：每个数是上一行相邻之和，比如 15 是 5+10，公式如下

$a[i][j] = a[i-1][j-1] + a[i-1][j]$ 如下图，抽象为二维数组

$a[5][2] = a[4][1] + a[4][2]$ $10 = 4 + 6$

```

1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
1 5 10 10 5 1
1 6 15 20 15 6 1
1 7 21 35 35 21 7 1
1 8 28 56 70 56 28 8 1
1 9 36 84 126 126 84 36 9 1
```

边数为数字 1，不满足公式，看似特殊，可以对其稍加修改，得到如下

```

0 1 0
0 1 1 0
0 1 2 1 0
0 1 3 3 1 0
0 1 4 6 4 1 0
0 1 5 10 10 5 1 0
0 1 6 15 20 15 6 1 0
0 1 7 21 35 35 21 7 1 0
0 1 8 28 56 70 56 28 8 1 0
0 1 9 36 84 126 126 84 36 9 1 0
```

这样，用二维数组装这些数据，中间的数字部分就全部满足上述公式了

行是 n, 列是 n+2

```
#define H 9    //二维数组行
#define L H+2  //二维数组列
```



```

int main(void)
{
    int arr[H][L] = { 0 }, i, j;    //初始化 0
    arr[0][1] = 1;                  //第一行的 1 先赋值
    for (i = 1; i < H; i++)          //从第二行开始
        for (j = 1; j < i + 1; j++)
            arr[i][j] = arr[i - 1][j - 1] + arr[i - 1][j];
    return 0;
}

```

数据就有了，接下来输出二维数组就行了

```

    for (i = 0; i < H; i++)
    {
        for (j = 1; j <= i+1; j++)
            printf("%4d", arr[i][j]);
        putchar('\n');
    }

```

```

1
1  1
1  2  1
1  3  3  1
1  4  6  4  1
1  5 10 10  5  1
1  6 15 20 15  6  1
1  7 21 35 35 21  7  1
1  8 28 56 70 56 28  8  1

```

然后加上空格

```

    for (i = 0; i < H; i++)
    {
        for (j = i; j < H; j++)
            printf("  ");    //两个空格
        for (j = 1; j <= i+1; j++)
            printf("%4d", arr[i][j]);
        putchar('\n');
    }

```

蛇形结构，环形打印，如下

```
1  2  3  4  5  6
20 21 22 23 24 7
19 32 33 34 25 8
18 31 36 35 26 9
17 30 29 28 27 10
16 15 14 13 12 11
```

规则：蛇形



书写思路：利用二维数组，一圈是四个边，仅需要记录每个边的起始坐标，和终点坐标，就可以用循环分别遍历四条边，进行赋值。

代码逻辑：

```
#define N 6
int x1 = 0, y1 = 0; //起点坐标
int x2 = 0, y2 = 0; //终点坐标
int shu = 1;        //数字
int n = 0; //表示每一圈的起始坐标点 第一圈 0,0 第二圈 1,1 第三圈 2,2
```

上边设置代码

```
x1 = n, y1 = n; //起点坐标
x2 = x1, y2 = N - 1 - n; //终点坐标 N是数组行数
//设置上行
int hang = x1, lie = y1;
for (lie = y1; lie <= y2; lie++)
    arr[hang][lie] = shu++;
```

右边设置代码

```
x1 = n+1, y1 = lie - 1; //起点坐标
x2 = N - 1 - n, y2 = y1; //终点坐标
//设置上行
hang = x1, lie = y1;
for (hang = x1; hang <= x2; hang++)
    arr[hang][lie] = shu++;
```

下边设置代码

```
x1 = hang - 1, y1 = lie - 1; //起点坐标
x2 = x1, y2 = n; //终点坐标
//设置上行
hang = x1, lie = y1;
for (lie = y1; lie >= y2; lie--)
    arr[hang][lie] = shu++;
```

左边设置代码

```
x1 = hang-1, y1 = lie + 1; //起点坐标
x2 = n+1, y2 = y1; //终点坐标
//设置上行
hang = x1, lie = y1;
for (hang = x1; hang >= x2; hang--)
    arr[hang][lie] = shu++;
```

以上是一圈的代码，要很多圈，循环如下

```
while (shu <= N * N) //有多少元素，shu 加多少次
{
    n += 1; //第 2, 3, 4, 5 圈
}
```

ABCD

99 乘法表，四个角的

```
1x1= 1  1x2= 2  1x3= 3  1x4= 4  1x5= 5  1x6= 6  1x7= 7  1x8= 8  1x9= 9
2x1= 2  2x2= 4  2x3= 6  2x4= 8  2x5=10  2x6=12  2x7=14  2x8=16  2x9=18
3x1= 3  3x2= 6  3x3= 9  3x4=12  3x5=15  3x6=18  3x7=21  3x8=24  3x9=27
4x1= 4  4x2= 8  4x3=12  4x4=16  4x5=20  4x6=24  4x7=28  4x8=32  4x9=36
5x1= 5  5x2=10  5x3=15  5x4=20  5x5=25  5x6=30  5x7=35  5x8=40  5x9=45
6x1= 6  6x2=12  6x3=18  6x4=24  6x5=30  6x6=36  6x7=42  6x8=48  6x9=54
7x1= 7  7x2=14  7x3=21  7x4=28  7x5=35  7x6=42  7x7=49  7x8=56  7x9=63
8x1= 8  8x2=16  8x3=24  8x4=32  8x5=40  8x6=48  8x7=56  8x8=64  8x9=72
9x1= 9  9x2=18  9x3=27  9x4=36  9x5=45  9x6=54  9x7=63  9x8=72  9x9=81
```

```
int i, j;
for (i = 1; i <= 9; i++)
{
    for (j = 1; j <= 9; j++)
        printf("%dx%d=%2d  ", i, j, i * j);
    putchar('\n');
}
```

左下

```
1x1= 1
1x2= 2  2x2= 4
1x3= 3  2x3= 6  3x3= 9
1x4= 4  2x4= 8  3x4=12  4x4=16
1x5= 5  2x5=10  3x5=15  4x5=20  5x5=25
1x6= 6  2x6=12  3x6=18  4x6=24  5x6=30  6x6=36
1x7= 7  2x7=14  3x7=21  4x7=28  5x7=35  6x7=42  7x7=49
1x8= 8  2x8=16  3x8=24  4x8=32  5x8=40  6x8=48  7x8=56  8x8=64
1x9= 9  2x9=18  3x9=27  4x9=36  5x9=45  6x9=54  7x9=63  8x9=72  9x9=81
```

```
int i, j;
for (i = 1; i <= 9; i++)
{
    for (j = 1; j <= 9; j++)
        if (i >= j) //条件
            printf("%dx%d=%2d  ", j, i, i * j);
    putchar('\n');
}
```

右下

```

                                     1x1= 1
                                     1x2= 2  2x2= 4
                                     1x3= 3  2x3= 6  3x3= 9
                                     1x4= 4  2x4= 8  3x4=12  4x4=16
                                     1x5= 5  2x5=10  3x5=15  4x5=20  5x5=25
                                     1x6= 6  2x6=12  3x6=18  4x6=24  5x6=30  6x6=36
                                     1x7= 7  2x7=14  3x7=21  4x7=28  5x7=35  6x7=42  7x7=49
                                     1x8= 8  2x8=16  3x8=24  4x8=32  5x8=40  6x8=48  7x8=56  8x8=64
1x9= 9  2x9=18  3x9=27  4x9=36  5x9=45  6x9=54  7x9=63  8x9=72  9x9=81
```

```
for (i = 1; i <= 9; i++)
{
    for (j = 9; j > i; j--)
        printf(" ");
}
```

```

for (j = 1; j <= 9; j++)
    if (i >= j)
        printf("%dx%d=%2d  ", j, i, i * j);
    putchar('\n');
}

```

左上

```

1x1= 1  1x2= 2  1x3= 3  1x4= 4  1x5= 5  1x6= 6  1x7= 7  1x8= 8  1x9= 9
2x2= 4  2x3= 6  2x4= 8  2x5=10  2x6=12  2x7=14  2x8=16  2x9=18
3x3= 9  3x4=12  3x5=15  3x6=18  3x7=21  3x8=24  3x9=27
4x4=16  4x5=20  4x6=24  4x7=28  4x8=32  4x9=36
5x5=25  5x6=30  5x7=35  5x8=40  5x9=45
6x6=36  6x7=42  6x8=48  6x9=54
7x7=49  7x8=56  7x9=63
8x8=64  8x9=72
9x9=81

```

```

for (i = 1; i <= 9; i++)
{
    for (j = 1; j <= 9; j++)
        if (i <= j)
            printf("%dx%d=%2d  ", i, j, i * j);
    putchar('\n');
}

```

右上

```

1x1= 1  1x2= 2  1x3= 3  1x4= 4  1x5= 5  1x6= 6  1x7= 7  1x8= 8  1x9= 9
          2x2= 4  2x3= 6  2x4= 8  2x5=10  2x6=12  2x7=14  2x8=16  2x9=18
                3x3= 9  3x4=12  3x5=15  3x6=18  3x7=21  3x8=24  3x9=27
                    4x4=16  4x5=20  4x6=24  4x7=28  4x8=32  4x9=36
                        5x5=25  5x6=30  5x7=35  5x8=40  5x9=45
                            6x6=36  6x7=42  6x8=48  6x9=54
                                7x7=49  7x8=56  7x9=63
                                    8x8=64  8x9=72
                                        9x9=81

```

```

int i, j;
for (i = 1; i <= 9; i++)
{
    for (j = 1; j < i; j++)
        printf(" ");
    for (j = 1; j <= 9; j++)
        if (i <= j)
            printf("%dx%d=%2d  ", i, j, i * j);
    putchar('\n');
}

```

15 整数翻转

将任意 int 整数，如 123456，转成 654321

思路：

1、将数位从个位一个一个拆出来，分别拆出 6,5,4,3,2,1

```
int az = 123456, af = 0;
while (az != 0) //az 为: 123456, 12345, 1234, 123, 12, 1, 0, 退出循环
{
    int w = az % 10; //w 装的依次是 6, 5, 4, 3, 2, 1
    az /= 10;        //变化为: 12345, 1234, 123, 12, 1, 0
}
```

2、将得到的各个位累加到一起

```
int az = 123456, af = 0;
while (az != 0)
{
    int w = az % 10;
    af = w + 10 * af; //累加
    az /= 10;
}
```

//代码的执行过程

```
//1、af = 6 + 10 * 0      af == 6
//2、af = 5 + 10 * 6      af == 65
//3、af = 4 + 10 * 65     af == 654
//4、af = 3 + 10 * 654    af == 6543
//5、af = 2 + 10 * 6543   af == 65432
//6、af = 1 + 10 * 65432  af == 654321
//结束循环，af == 654321
```

最终代码

```
#include<stdio.h>
int main()
{
    int az = 123456, af = 0;
    printf("翻转前: %d\n", az);
    while (az != 0)
    {
        int w = az % 10;
        af = w + 10 * af; //累加
        az /= 10;
    }
    az = af;
    printf("翻转后: %d\n", az);
    return 0;
}
```

16 水仙花

所有 3 位数中，该数的各个位的立方和等于该数，称其为水仙花数，请找到

```
#include<stdio.h>
int main()
{
    int i = 0, ge, shi, bai;
    for (i = 100; i <= 999; i++)
    {
        ge = i % 10;          //个位
        shi = i / 10 % 10;    //十位
        bai = i / 100;        //百位
        if (ge * ge * ge + shi * shi * shi + bai * bai * bai == i)
            printf("%d ", i);
    }
    return 0;
}
```

//立方处不可以 ge^3 这种写法，这是亦或位运算，没有立方的作用

16 水仙花

所有 3 位数中，该数的各个位的立方和等于该数，称其为水仙花数，请找到

```
#include<stdio.h>
int main()
{
    int i = 0, ge, shi, bai;
    for (i = 100; i <= 999; i++)
    {
        ge = i % 10;          //个位
        shi = i / 10 % 10;    //十位
        bai = i / 100;        //百位
        if (ge * ge * ge + shi * shi * shi + bai * bai * bai == i)
            printf("%d ", i);
    }
    return 0;
}
```

//立方处不可以 ge^3 这种写法，这是亦或位运算，没有立方的作用

17 整数是否回文

回文就是对称相同

3 、 123321 、 12321

逻辑：

方法 1、先翻转，看反转后的数与之前是否一样

```
#include<stdio.h>
int main()
{
    int az = 123321, w, af = 0;
    int n = az; //中间变量，用来遍历 az
    while (n != 0)
    {
        w = n % 10;
        af = af * 10 + w;
        n /= 10;
    }
    if (az == af)
        printf("%d 是回文\n", az);
    else
        printf("%d 不是回文\n", az);
    return 0;
}
```

方法 2、装字符串里，判断字符串是否回文，没有必要了，方法 1 最好

18 素数(质数)

输出所有 3 位数中的素数(质数)，并计算他们的和

素数：只能被 1 与自身整除的数，2 是质数，0 与 1 既不是素数也不是合数

思路逻辑：

判断一个数 n 是不是素数，那么就用 n 分别去除 $2 \sim n-1$ ，如果能被这个范围内的数整除，那么 n 就不是素数。

```
int prime(int n)
{
    if (0 == n || 1 == n)
        return 0; //不是素数
    if (2 == n)
        return 1; //是素数 返回 n 也行
    int j;          //老板本编译器，放前面
    for (j = 2; j <= n-1; j++)
        if (n % j == 0) //非素数
            return 0;
    return 1; //经过循环之后，一定是素数
}

int main()
{
    int n, j, sum = 0;
    for (n = 100; n <= 999; n++)
    {
        if (0 != prime(n)) //是素数
        {
            sum += n; //累加
            printf("%d ", n);
        }
    }
    printf("\n sum = %d\n", sum);
}
```

变化部分：

素数循环条件判断处： $j < n / 2$

```
int m = n / 2;
```

```
for (j = 2; j <= m; j++) //效率更好
```

```
for (j = 2; j <= n-1; j++) //也可以，就是浪费很多时间
```

```
int m = (int)sqrt(n);
```

```
for (j = 2; j <= m; j++) //循环次数更少，效率更好
```

//数学分析

//比如 $49 = 7 * 7$ 如果还有两个数相乘得 49，一定是其中一个比 7 小，其中一个比 7 大，所以只需在 $2 \sim 7$ 之间找整除的数即可

19 质因子

找出指定数的质因子。(所有因子中的质数找出来)

因子也叫因数，比如 $3 \times 4 = 12$ ，3 4 就是 12 的因数也叫因子

另外：1*12、2*6、3*4 都得 12，所以 12 的因子有 1,2,3,4,6,12

其中，2,3 是质数，所以本题目找到 2,3

整数 n 的因子，用 n 分别对 1~sqrt(n) 的数除运算，如果整除，那就是因子

```
int n = 12, j;
for (j = 1; j <= n; j++)
{
    if (0 == n % j) //是因子
    {

    }
}
```

得出一个因子，就对其进行质数判断

```
int prime(int n)
{
    if (0 == n || 1 == n)
        return 0; //不是素数
    if (2 == n)
        return 1; //是素数 返回 n 也行
    int j, m = n/2;
    for (j = 2; j <= m; j++)
        if (n % j == 0) //非素数
            return 0;
    return 1; //经过循环之后，一定是素数
}
```

```
int main(void)
{
    int n = 12, j;
    for (j = 1; j <= n; j++)
    {
        if (0 == n % j) //是因子
        {
            if (0 != prime(j)) //是素数
                printf("%d ", j);
        }
    }
}
```

20 最大公约数

编程得到两个数的最大公约数

最大公因数、最大公因子，即两个数的因数中，相同的，且最大的那个

12 的因数 1 2 3 4 6 12

20 的因数 1 2 4 5 10 20

相同的因数 1 2 4 最大公约数 4

编码思路：

1、找到两个数的公约数

```
int n1 = 12, n2 = 20;
int n = n1 > n2 ? n2 : n1; //n 装两个数中最小的
int j;
for (j = 1; j <= n; j++) //<= 两个数中小的那个
{
    if (0 == n1 % j && 0 == n2 % j) //公因子,
    {
        printf("%d ", j); //输出所有公因子
    }
}
```

2、找出最大的

```
int n1 = 12, n2 = 20;
int n = n1 > n2 ? n2 : n1;
int j, maxg = 0;
for (j = 1; j <= n; j++) { //<= 两个数中小的那个

    if (0 == n1 % j && 0 == n2 % j) //公因子,
    {
        if (j > maxg) //找最大值
            maxg = j;
    }
}
printf("最大: %d", maxg);
```

更好的代码

```
int n1 = 12, n2 = 20;
int n = n1 > n2 ? n2 : n1;
int j;
for (j = n; j >= 1; j--)
    if (n1 % j == 0 && n2 % j == 0)
        break;
printf("最大 : %d ", j);
```

21 最小公倍数

找到两个数的最小公倍数

思路：先确定是大的那个数的倍数，在确定小的，效率高

比如：12,20 公倍数 n 查找数为 20,40,60,80,100..... n+=20

因为 24 36 这种一定不是 20 的倍数

```
int n1 = 12, n2 = 20;
int maxn = n1 > n2 ? n1 : n2; //找到大的那个
int n = maxn;
while (1) //不知道找多少次，所以死循环
{
    if (n % n1 == 0 && n % n2 == 0) //用大的，先是大的倍数
        break;
    n += maxn;
}
printf("最大: %d", n);
```

22 完全数

找到 100 以内的小完数

完数：一个数的所有的真因子（即除了自身以外的约数）的和，恰好等于它自身

1 不是完数

```
int n = 2, j, sum = 0; //n 是要遍历的数
while (n<=100)
{
    sum = 0; //每次都要重新置 0
    for (j = 1; j < n; j++) //sqrt(n)
        if (n % j == 0)
            sum += j;
    if (sum == n)
        printf("%d ", n);
    n++;
}

int n = 2;
//循环2~100
while (n <= 1000)
{
    //累加真因子
    int sum = 0, j = 1;
    int nn = sqrt(n);
    for (j = 2; j <= nn; j++)
        if (n % j == 0)
        {
            sum += j;
            if (j != n/j)
                sum += n / j;
        }

    sum += 1;
    //判断完数
    if (sum == n)
        printf("%d 是完数\n", n);
    n++;
}
```

23 数组断回文

以数组辅助判断整数是否回文

回文就是对称相同

3 、 123321 、 12321

以数组辅助

1、将整数各个位依次拆下来，装进数组中

```
int a = 123321, aa = a; //aa 记录原值
int arr[10] = { 0 }, in = 0; //in 是数组下标，arr10 元素就可以
while (a)
{
    arr[in] = a % 10; //取低位
    in++;           //下标++
    a /= 10;        //自除 10
}
```

2、对称比较数组的各个元素

//循环结束 in 指向最后一个元素的下一个位置

```
in -= 1; //指向最后一个元素
int i = 0; //首元素下标
while (i < in)
{
    if (arr[i] != arr[in]) //找到不相等
        break;
    i++; //一个从前往后走
    in--; //一个从后往前走
}
```

3、判断是否回文

循环结束有两个方式

条件结束: $i \geq in$ 此时没走 break, 所以回文

break 结束: 有对应元素不相等, 不回文

```
if (i >= in)
{
    printf("%d 回文\n", aa);
}
else
{
    printf("%d 不回文\n", aa);
}
```

24 完全平方数

题目描述：一个整数，它加上 100 后是一个完全平方数，再加上 168 又是一个完全平方数：开方之后是整数

1、完全平方判断方法

```
int n = 12;
int x = (int)sqrt(n); //开平方
if (x * x == n)        //完全平方
    printf("%d 是完全平方数", n);
```

封装一下

```
int IsSqrt(int n)
{
    int x = (int)sqrt(n);
    if (x * x == n)
        return 1; //是完全平方数
    return 0;     //非完全平方数
}
```

2、遍历范围：假定 10000

```
int n = 0;
for (n = 0; n < 10000; n++)
{
    if (1 == IsSqrt(n + 100) && 1 == IsSqrt(n + 268))
    {
        printf("%d 是完全平方数\n", n);
    }
}
```

边界推断：

$$x * x == n + 100$$

$$y * y == n + 268$$

由于 $n+100$ 与 $n+268$ 差值是 168

所以 $y * y - x * x == 168$ 差值最大 168

假设 $y=168$ $x=167$ 两个相邻的差值最小，最小的要小于等于 168，就是我们遍历的边界点， y x 中间隔得越多，差值越大

$$28224 - 27889 == 335 > 168 \text{ 所以边界 } 28224 \text{ 不行 } 168*168 \text{ 太大了}$$

$$100*100 - 99*99 == 199 > 168$$

$$85*85 - 84*84 == 169 \text{ 所以 } 85*85 \text{ 可以作为边界 } 7225 \text{ 所以 } n < 7225$$

25 $2+22+222+2222+22222$

逻辑:

$2 \quad 22(2*10+2) \quad 222(22*10+2) \quad 2222(222*10+2)$

$n = (n - 1) * 10 + 2 \quad n=0$

第1次循环: $n = 0*10 + 2 == 2$

第2次循环: $n = 2*10 + 2 == 22$ 用上一次的结果

第3次循环: $n = 22*10 + 2 == 222$

代码:

```
int n = 0, i = 1, sum = 0;
for (i = 1; i <= 5; i++)
{
    n = n * 10 + 2; //2 22 222 2222 22222
    sum += n;      //累加
    printf("%d + ", n); //输出 2 + 22 + 222 +
}
n = n * 10 + 2;    //再算一次
printf("%d = %d", n, sum); //可以盖住一个+
```

26 $1! + 2! + 3! + 4! + 5!$

5!表示阶乘, 即 $5*4*3*2*1$

思路:

$5! = 4! * 5 \quad 4! = 3! * 4 \quad 3! = 2! * 3 \quad 2! = 1! * 2 \quad 1! = 1$

所以:

```
int n = 1, i = 1, sum = 0;
for (i = 1; i <= 5; i++)
{
    n *= i; //1! 2! 3! 4! 5!
    sum += n; //求和
    printf("%d! == %d\n", i, n);
}
printf("和 == %d", sum);
```

或者

```
int n = 1, i = 1, sum = 0;
for (i = 1; i <= 4; i++)
{
    n *= i;
    sum += n;
    printf("%d! + ", i);
}
n *= i;
sum += n;
printf("%d! == %d", i, sum);
```

27 斐波拉契数列 非递归 到递归专题会讲递归形式

0 1 1 2 3 5 8 13 21.... $n = (n-1) + (n-2)$

即第 n 项的值等于第 $n-1$ 项加第 $n-2$ 项

算法逻辑:

定义数组 `int a[47] = {0,1};`

数组用来装斐波拉契的项

数组是最多装 47 项, 因为第 48 项开始, 就超过了 `int` 的可表示范围了

如果想得到更多的斐波拉契项, 使用更大的 `long long`, 如果还要更多, 就要大数运算的算法了

初始化前两项, 从第 3 项开始使用公式对赋值数组,

代码: 输出 `int` 可表示的所有项

```
int main(void)
{
    int a[47] = {0,1}, i = 1;
    printf("0 1 ");
    for (i = 2; i <= 46; i++)
    {
        a[i] = a[i-1] + a[i-2];
        printf("%d ", a[i]);
    }
}
```

代码: 输出指定项, 比如输入第 10 项

```
int main(void)
{
    int a[47] = {0,1}, i = 1;
    int n = 10;
    for (i = 2; i < n; i++)
    {
        a[i] = a[i-1] + a[i-2];
    }
    printf("%d ", a[i-1]); //循环结束 i 为 10 第 11 项
}
```


28 根据公式编程计算得到 π

$$\sum_{n=0}^{\infty} \frac{4^n n! (1103 + 26390 * n)}{(n!)^4 * 396^{4*n}}$$

得到的结果是 sum

```
 $\pi$  = 1.0 / (sum * xishu);  
xishu = 2.0 * sqrt(2) / 9801.0; 0 != 1  
这种类型的题目，看起来很复杂，其实很简单  
代码思路：
```

将公式各个部分拆解出来，单独计算
然后再合起来

系数

```
double xishu = 2.0 * sqrt(2) / 9801.0;
```

求和循环

```
int n = 1; //从1开始，0没有意义  
for (n = 1; n < 10; n++) //10不定，越多越精确  
{  
}
```

4^n 写循环里

```
double fn = pow(4, n);
```

$n!$ 写循环里

```
int nj = 1, j; //可以用 double  
for (j = 1; j <= n; j++)  
    nj *= j;
```

$1103 + 26390 * n$ 写循环里

```
double cn = 1103 + 26390 * n;
```

$(n!)^4$ 写循环里

```
double nj4 = pow(nj, 4);
```

396^{4*n} 写循环里

```
double n396 = pow(396, 4 * n);
```

求和

```
sum += (fn * nj * cn) / (nj4 * n396);  
printf("pai = %.101f", 1.0 / (sum * xishu));
```

29 爬楼梯问题

假设你正在爬楼梯。需要 n 阶你才能到达楼顶。

每次你可以爬 1 或 2 个台阶。你有多少种不同的方法可以爬到楼顶呢？

注意：给定 n 是一个正整数。

1 阶楼梯有 1 种爬法, 1

2 阶楼梯有 2 种爬法, 1+1, 2

3 阶楼梯有 3 种爬法, 1+1+1, 1+2, 2+1

4 阶楼梯有 5 种爬法, 1+1+1+1, 1+1+2, 1+2+1, 2+1+1, 2+2

满足斐波拉契数列, 所以可用斐波拉契规律

前面讲的斐波拉契使用了数组, 这个就不用数组了

```
#include <stdio.h>
int main()
{
    int f1 = 1, f2 = 2, f3; //前两项
    printf("1 2 ");
    int i = 0, n = 10;
    for (i = 3; i <= n; i++)
    {
        f3 = f2 + f1; //
        f1 = f2;      //f2 变新 f1
        f2 = f3;      //f3 变新 f2
        printf("%d ", f3);
    }
    return 0;
}
```

30 经典数列求和

$$-\frac{1}{3*1} + \frac{3}{5*2} - \frac{6}{7*6} + \frac{10}{9*24} - \frac{15}{11*120} + \frac{21}{13*720} \dots (-1)^n * \frac{1+2+3+\dots+n}{(2*n+1)*n!}$$

`n >= 1`

思路：将通项公式拆分，然后组合

1、求和循环，`n` 从 1 开始，10 结束

```
int n = 1;
for (n = 1; n <= 10; n++) //求前 10 项和
{

}
```

2、正负号：用累乘-1 实现正负号切换

```
int n = 1, fh = 1;
for (n = 1; n <= 10; n++) //求前 10 项和
{
    fh *= -1; // - + - + - + - +
    //如果 fh 初始值是-1，则呈现 + - + - + - + -
}
```

3、分子：当前 `n` 到 1 的累加

```
int n = 1, fz = 0;
for (n = 1; n <= 10; n++) //求前 10 项和
{
    fz += n; //分别是 1 3 6 10 15 21
}
```

4、分母：前部分 `2*n+1`

```
int n = 1, fm1 = 0;
for (n = 1; n <= 10; n++) //求前 10 项和
{
    fm1 = 2 * n - 1;
}
```

5、分母：后部分，`n` 到 1 的累乘

```
int n = 1, fm2 = 1;
for (n = 1; n <= 10; n++) //求前 10 项和
{
    fm2 *= n;
}
```

7、累加：合起来

```
sum += (double)(fh * fz) / (double)(fm1 * fm2);
```

一定不要忘记将四个 `int` 值转成 `double`，某些人经常忘记

递归做数列的口诀

- 1、找起始项
- 2、找通项公式
- 3、函数有返回值，参数做变化

31 累加

数列1、3、6、10、15.....用递归求第8项

(1)、规律

$$f(1) = 1$$

$$f(2) = 2+1 = 2+f(1)$$

$$f(3) = 3+2+1 = 3+f(2)$$

$$f(4) = 4+3+2+1 = 4+f(3)$$

$$f(5) = 5+4+3+2+1 = 5+f(4)$$

$$f(n) = n+f(n-1) \quad \text{通项公式}$$

代码：

```
int f(int n)
{
    if (1 == n)           //固定值项为递归跳出
        return 1;
    return n + f(n - 1);  //return 通项公式
}
```

递归做数列的口诀

- 1、找起始项：递归出口
- 2、找通项公式：递归本体
- 3、函数有返回值，参数做变化 `int fun(int n)`

32、阶乘

数列1!,2!,3!,4!,5!,6!,7!.....每一项是项数的阶乘，用递归求出第8项的值

规律

$$f(1) = 1$$

$$f(2) = 2*1 = 2*f(1)$$

$$f(3) = 3*2*1 = 3*f(2)$$

$$f(4) = 4*3*2*1 = 4*f(3)$$

$$f(5) = 5*4*3*2*1 = 5*f(4)$$

$$f(n) = n*f(n-1) \quad \text{通项公式}$$

代码：

```
int f(int n)
{
```

```

    if (1 == n)          //固定值项为递归跳出
        return 1;
    return n * f(n - 1); //return 通项公式
}

```

递归做数列的口诀

- 1、找起始项：递归出口
- 2、找通项公式：递归本体
- 3、函数有返回值，参数做变化：函数头 int fun(int n)

33 斐波拉契

输出斐波拉契数列第 8 项

0 1 1 2 3 5 8 13 21 34.... 从哪开始都行

$f(1) = 0$ //也可从 1 开始

$f(2) = 1$ //前两项固定

$f(3) = 1 + 0 = f(2) + f(1)$

$f(4) = 2 + 1 = f(3) + f(2)$

$f(5) = 3 + 2 = f(4) + f(3)$

$f(6) = 5 + 3 = f(5) + f(4)$

$f(7) = 8 + 5 = f(6) + f(5)$

$f(8) = 13 + 8 = f(7) + f(6)$

$f(n) = f(n-1) + f(n-2)$

代码：

```

int f(int n)
{
    if (1 == n)          //固定值项为递归跳出
        return 0;
    if (2 == n)          //固定值项为递归跳出
        return 1;
    return f(n - 2) + f(n - 1); //return 通项公式
}

```

34、求勒让德多项式的

值公式如下：

$$\text{fn}(x) \begin{cases} \text{fn}(x) = 1 & (n = 0) \\ \text{fn}(x) = x & (n = 1) \\ \text{fn}(x) = ((2n-1)x - \text{fn}(n-1, x) - (n-1)\text{fn}(n-2, x))/n & (n > 1) \end{cases}$$

这是一种题型，前两项 $n=0$ $n=1$ 为固定项，出递归

x 是未知数，是一个可传递值，可以通过参数传递

```
double fn(int n, double x)
```

```
{
    if (0 == n)
        return 1.0;
    if (1 == n)
        return x;
    return ((2 * n - 1) * x - fn(n - 1, x) - (n - 1) * fn(n - 2, x)) / n;
}
```

```
int main()
```

```
{
    printf("%lf", fn(10, 3.4));
    return 0;
}
```

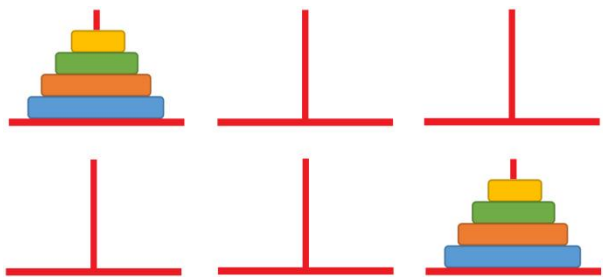
35 汉诺塔

将左柱上的多层塔，移动到右柱上

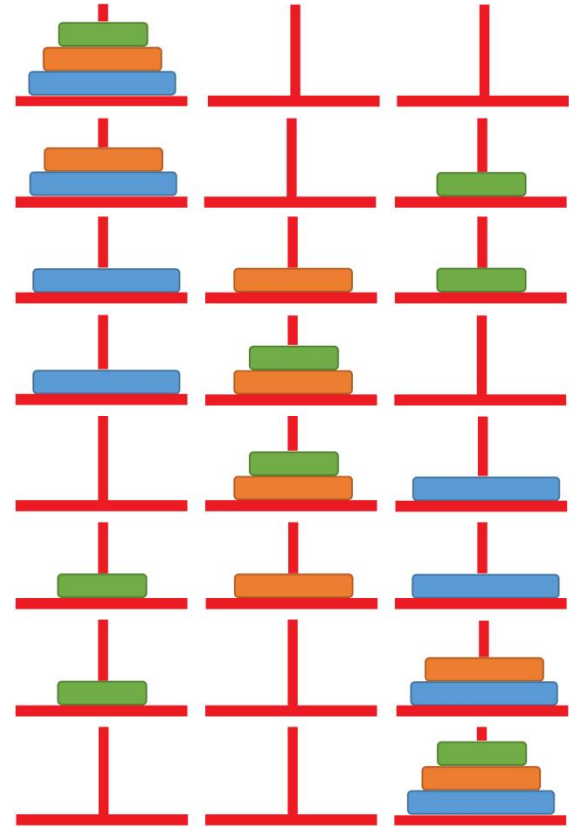
要求：

每次只能移动一层塔

短塔必须在长塔的上面



过程展示



1	L->R
2	L->M, L->R, M->R
3	L->R, L->M, R->M, L->R, M->L, M->R, L->R
4	L->M, L->R, M->R, L->M, R->L, R->M, L->M, L->R, M->R, M->L, R->L, M->R, L->M, L->R, M->R

1	L->R
2	L->M, L->R, M->R
3	L->R, L->M, R->M, L->R, M->L, M->R, L->R
4	L->M, L->R, M->R, L->M, R->L, R->M, L->M, L->R, M->R, M->L, R->L, M->R, L->M, L->R, M->R

对于 n 层塔的移动步数

公式

$$f(n) \begin{cases} f(n) = 1 & (n=1) \\ f(n) = 2*f(n-1)+1 & (n>1) \end{cases}$$

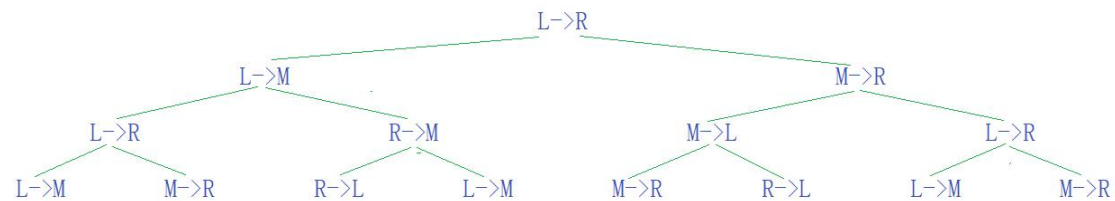
根据公式得到 n 层汉诺塔移动次数的递归代码如下：

```
int f(int n)
{
    if (1 == n)
        return 1;
    return 2 * f(n - 1) + 1;
}
```

汉诺塔移动过程的递归代码分析如下：

1	L->R					
2	L->M,		L->R,		M->R	
3	L->R,	L->M,	R->M,	L->R,	M->L,	M->R, L->R
4	L->M, L->R, M->R, L->M, R->L, R->M, L->M, L->R, M->R, M->L, R->L, M->R, L->M, L->R, M->R					

根据图标，可以得到 n 层汉诺塔，他的过程满足二叉树的中序遍历规则抽象如下：



中序遍历的顺序即是 4 层汉诺塔移动顺序

1、满足二叉树结构，所以双递归结构

```
int f(int n)
{
    f(n - 1);
    f(n - 1);
}
```

2、遍历顺序为中序，所以输出在双递归之间

```
int f(int n)
{
    f(n - 1);
    printf("输出");
    f(n - 1);
}
```

3、参数分析，返回值不使用，所以用 void 即可

三根塔柱，L M R

左子树：L->R L->M L->R L->M.....

右子树: L->R M->R L->R M->R.....

```
int f(int n, char L, char M, char R)
{
    f(n - 1, L, R, M); //左子树
    printf("输出");
    f(n - 1, M, L, R); //右子树
}
```

4、递归出口, 输出

```
int f(int n, char L, char M, char R)
{
    if (1 == n) //一层, 直接挪
        printf("%c->%c", L, R);
    else
    {
        f(n - 1, L, R, M); //左子树
        printf("%c->%c", L, R);
        f(n - 1, M, L, R); //右子树
    }
}
```

37 利用递归倒序输出整数各个位

题目：整数 12345，利用递归输出 5 4 3 2 1

```
void f(int n)
{
    if (0 != n)
    {
        printf("%d ", n % 10);
        f(n / 10);
    }
}

int main()
{
    f(12345);
    return 0;
}
```

先给代码，然后讲解过程

```
void f(12345)
{
    if (0 != n)
    {
        printf("%d ", n % 10); 5
        f(n / 10); ➡ void f(1234)
    }
    {
        if (0 != n)
        {
            printf("%d ", n % 10); 4
            f(n / 10); ➡ void f(123)
        }
        {
            if (0 != n)
            {
                printf("%d ", n % 10); 3
                f(n / 10); ➡ void f(12)
            }
            {
                if (0 != n)
                {
                    printf("%d ", n % 10); 2
                    f(n / 10); ➡ void f(1)
                }
                {
                    if (0 != n)
                    {
                        printf("%d ", n % 10); 1
                        f(n / 10);
                    }
                }
            }
        }
    }
}
```

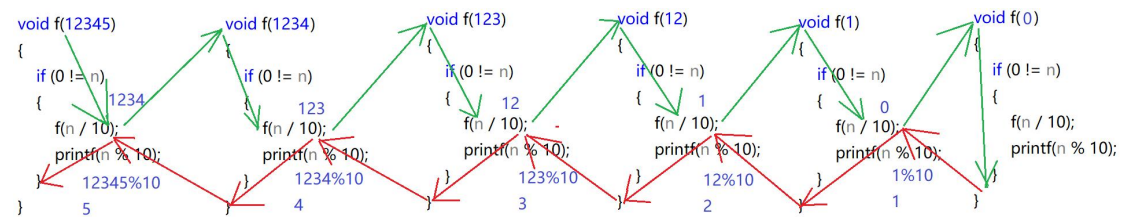
38 利用递归正序输出整数各个位

题目：整数 12345，利用递归输出 1 2 3 4 5

```
void f(int n)
{
    if (0 != n)
    {
        //printf("%d ", n % 10); //倒序
        f(n / 10);
        printf("%d ", n % 10); //正序
    }
}

int main()
{
    f(12345);
    return 0;
}
```

给出代码执行过程



递归逻辑执行过程如上

每次递归调用相当于再次执行此函数，函数执行完跳回调用处，继续往下执行

39 使用递归计算字符串长度

递归代码

```
int len(char *p)
{
    if (*p == 0)
        return 0;
    return len(p + 1) + 1;
}

int main()
{
    printf ("%d", len("abc"));
    return 0;
}
```

使用递归

规律：

- 1、字符串结尾\0 结束计算
长度不算\0，所以\0长度为0
- 2、比如字符串abcd\0
以a为首的长度是bcd\0+1
以b为首的长度是cd\0+1
以c为首的长度是d\0+1
以d为首的长度是\0+1

1、函数头

```
int len(char *p) //传递字符串首地址
{
}
```

2、字符串长度不算结尾 \0

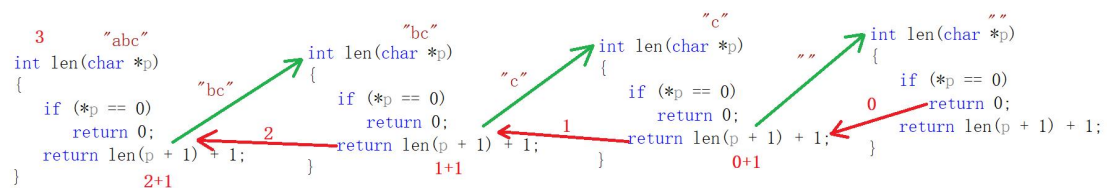
```
if (*p == 0) //递归出口
    return 0;
```

3、递归调用

```
return len(p + 1) + 1;
```

初始状态，p 是首地址，p+1 依次向后遍历

递归过程如下



40 最大值, 最小值

给定一数组 `int a[10] = { 3, 2, 6, 4, 7, 5, 3, 1, 7, 5 };`

编程得到一数组中的最大值与最小值, 并通过函数参数返回

步骤一: 得到最大值

步骤二: 得到最小值

步骤三: 传址调用传回最大值/最小值

函数头:

//参数 1: 数组首地址 参数 2: 数组长度 参数 3, 4: 传址调用

```
void get(char *arr, int len, int* max, int* min)
{
}
```

最大值 最小值

```
void get(char *arr, int len, int* max, int* min)
{
    int i;
    *max = arr[0], *min = arr[0];
    for (i = 1; i < len; i++)
    {
        if (*max < arr[i]) //找最大值
            *max = arr[i];
        if (*min < arr[i]) //找最小值
            *min = arr[i];
    }
}
```

函数调用

```
int main()
{
    int a[10] = { 3, 2, 6, 4, 7, 5, 3, 1, 7, 5 };
    int max = 0, min = 0;
    get(a, 10, &max, &min);
    printf("max = %d, min = %d", max, min);
    return 0;
}
```

41 翻转数组

给定一数组 `int a[10] = { 3, 2, 6, 4, 7, 5, 3, 1, 7, 5 };`

将数组元素前后调换位置, 调换后: 5,7,1,3,5,7,4,6,2,3

使用下标思路:

- 1、通过下标, 一个指前 $i=0$, 一个指后 $j=9$
- 2、循环, 交换 i, j 的元素 $t = a[i], a[i] = a[j], a[j] = t$
- 3、 $i++ \quad j--$
- 4、循环条件 $i < j$ 当 $i \geq j$ 时, 结束, 交换到中间元素了

```
int a[10] = { 3, 2, 6, 4, 7, 5, 3, 1, 7, 5 };
int i = 0, j = 9;
while (i < j)
{
    //交换
    int c = a[i];
    a[i] = a[j];
    a[j] = c;
    //指针变换
    i++, j--;
}
```

使用指针:

- 1、通过指针, 一个指前 $pq = \&a[0]$, 一个指后 $ph = \&a[9]$
- 2、循环, 交换 i, j 的元素 $t = *pq, *pq = *ph, *ph = t$
- 3、 $pq++ \quad ph--$
- 4、循环条件 $pq < ph$ 当 $pq \geq ph$ 时, 结束, 交换到中间元素了

```
int main(void)
{
    int a[10] = { 3, 2, 6, 4, 7, 5, 3, 1, 7, 5 };
    int* pq = &a[0], * ph = &a[9];
    while (pq < ph)
    {
        //交换
        int c = *pq;
        *pq = *ph;
        *ph = c;
        //指针变换
        pq++, ph--;
    }
    return 0;
}
```

42 用一个变量存 4 个字母

将字母 A B C D 存入一个 int 类型变量 e 中。

分析：

- 1、int 类型变量 e 有 4 字节空间
- 2、4 个字符，每个都是 1 字节可存
- 3、逻辑上将 e 作为 4 字节的 char 数组使用

```
int a = 0;    //四字节空间
char* p = (char*)&a; //将 a 空间做 4 字节 char 数组使用
p[0] = 'A';   //第 1 个字节
p[1] = 'B';   //第 2 个字节
p[2] = 'C';   //第 3 个字节
p[3] = 'D';   //第 4 个字节
//输出
int i;
for (i = 0; i < 4; i++)
    printf("%c ", ((char*)&a)[i]); //p[i]
```

4、同理：使用思路

```
double b;
//可以做 8 元素 char 数组
//可以做 4 元素 short 数组
//可以做 2 元素 int 数组
```

5、指针规则

(1)、*(p+i) == p[i] == i[p]

所以

((p+i)+j) == *(p[i]+j) == p[i][j]

(2)、p = &a *p == a

指针变量装着一个变量的地址，*+此指针就是该变量本身

所以上述代码

```
p[0] = 'A';   //第 1 个字节
p[1] = 'B';   //第 2 个字节
p[2] = 'C';   //第 3 个字节
p[3] = 'D';   //第 4 个字节
```

可以写成

```
*(p+0) = 'A';    *p = 'A';   //第 1 个字节
*(p+1) = 'B';   //第 2 个字节
*(p+2) = 'C';   //第 3 个字节
*(p+3) = 'D';   //第 4 个字节
```

43 交换两个变量

使用函数交换变量 a b 的值，传址调用

传址调用：

- 1、需要通过函数修改哪个变量，实参直接是&a
- 2、然后根据传递的地址类型，确定形参的类型

知识点：p = &a *p == a

指针变量装着一个变量的地址，*+此指针就是该变量本身

```
void swap(int* pa, int* pb)
{
    int c = *pa;
    *pa = *pb;
    *pb = c;
}

int main()
{
    int a = 2, b = 3;
    printf("%d %d", a, b);
    swap(&a, &b);
    printf("%d %d", a, b);
    return 0;
}
```

44 交换两个指针

使用函数交换指针变量 pa pb 的指向，传址调用

- 1、需要通过函数修改哪个变量，实参直接是&pa &pb
- 2、然后根据传递的地址类型，确定形参的类型

```
void swap(int** pa, int** pb)
{
    int *p = *pa;
    *pa = *pb;
    *pb = p;
}

int main()
{
    int a = 2, b = 3;
    int* pa = &a, *pb = &b;
    printf("%d %d", *pa, *pb);
    swap(&pa, &pb);
    printf("%d %d", *pa, *pb);
    return 0;
}
```


45、复杂指针

为以下变量 pf 赋值/初始化数据

```
int ((*pf)(int *))[3])(int *)
```

首先要解析 pf 的类型，把类型表述清楚，笔试面试遇到过

第一步：(*pf)先跟*结合，pf 是一个指针

第二步：(* (int *))该指针指向函数类型

第三步：该函数参数是一个 int*，返回值是一个指针 *

第四步：(*[3])该返回值指针指向 3 个元素的指针数组

第五步：int (int *)指针数组每个元素该函数类型指针

第六步：该函数一个 int*参数，返回值为 int 类型

反向定义类型指向

```
int fun1(int* p) { return 0; }
```

```
int fun2(int* p) { return 0; }
```

```
int fun3(int* p) { return 0; }
```

```
int (* pp[3])(int*) = {fun1, fun2, fun3};
```

```
int ((*fun(int* p))[3])(int*) { return &pp; }
```

```
int ((*(*pf)(int*)) [3])(int*) = &fun;
```

46、指针与二维数组

题目：有如下定义

```
int a[3][4] = { {1, 2, 3, 4},
                {5, 6, 7, 8},
                {9, 10, 11, 12} };
```

```
int* p1 = a[0];
```

```
int(*p2)[4] = a;
```

以下选项，分别指代的是什么

A: $*(p1 + 8) - 1$

B: $*(&p1[10] - 5) = 3$

C: $*((p2+1)[1]) = 4$

知识回顾：

1、数组涉及指针操作三定理

数组名即首元素首地址

指针加减 1，加减的值为指针类型的字节数大小

$*(p+1) == p[1]$

2、二维数组中，各表示法对应的内容如下

a 整个数组名字											
a[0]				a[1]				a[2] 每个小数组名字			
1	2	3	4	5	6	7	8	9	10	11	12
a[0][0]	a[0][1]	a[0][2]	a[0][3]	a[1][0]	a[1][1]	a[1][2]	a[1][3]	a[2][0]	a[2][1]	a[2][2]	a[2][3]
元素名字 变量名											

3、解析

A: $*(p1 + 8) - 1$

1、p1 是元素类型指针，装着首元素的地址，

2、p1+8 就是第九个元素的地址，

3、 $*(p1 + 8)$ 就是第九个元素，即 p1[8]

4、p1[8]-1 就是 9-1 结果是 8

B: $*(&p1[10] - 5) = 3$

1、p1 是元素类型指针，装着首元素的地址，

2、p1[10]就是第 11 个元素

3、&p1[10]就是第 11 个元素的地址

4、&p1[10] - 5 第 11 个元素的地址-5，就是第 6 个元素 6 的地址

5、*(&p1[10] - 5)就是第 6 个元素，p1[5]

6、=2,就是 p1[5]=2

C: *((p2+1)[1]) = 4

1、p2 是&a[0]，所以 p2+1 就是&a[1],第 2 个小数组的数组地址

2、(p2+1)[1]就是*(p2+1+1),即*&a[2],即 a[2],第 3 个小数组的名字，即元素 9 的地址，即&a[2][0]

(&a[1])[1]就是*(&a[1]+1),即*&a[2],即 a[2],第 3 个小数组的名字，即元素 9 的地址，即&a[2][0]

错误解析：&a[1][1] 加*就是a[1][1]

3、*((p2+1)[1])就是*&a[2][0]，也就是 a[2][0]

4、= 4，就是 a[2][0]=4

47、计算对角线的和

给定以下二维数组，完成函数，返回对角线的和

```
int a[3][3] = { {1, 2, 3},
                {4, 5, 6},
                {7, 8, 9}};
```

函数头：

```
int Sum(int(*arr)[3][3], int hang, int lie)
{
}
}
```

分析：

1、知识回顾

二维数组可以使用三种类型的指针访问

int* p1 = a[0]; //基本数据类型指针 &a[0][0]

p1: p1[0] p[1] p[2] ... p[8]

int(*p2)[3] = a; //一维数组类型指针 &a[0]

p2: p2[0][0] p2[0][1] ... p2[2][2]

int(*p3)[3][3] = &a; //二维数组类型指针

p3: (*p3)[0][0] (*p3)[0][1] ... (*p3)[2][2]

2、参数 1: int(*arr)[3][3]是二维数组指针，对二维数组名取地址得到的是二维数组地址，所以参数传递 &a

相当于: `int(*arr)[3][3] = &a;`

`arr == &a;`所以, `*arr == a;` 所以函数中访问二维数组元素的基本形式为`(*arr)[i][j]`

3、对角线下标规律

左斜对角线: 0,0 1,1 2,2 即 $i==j$

右斜对角线: 0,2 1,1 2,0 即 $i+j==2$

遍历数组, 将下标满足上而条件的元素累加即可

代码

```
int Sum(int(*arr)[3][3], int hang, int lie)
{
    int i, j, sum = 0;
    for (i = 0; i < hang; i++)
    {
        for (j = 0; j < lie; j++)
        {
            if (i == j || i + j == 2)
            {
                sum += (*arr)[i][j];
            }
        }
    }
    return sum;
}

int main(void)
{
    //以下代码的功能是输出: 你好, C3~
    int a[3][3] = { {1, 2, 3},
                    {4, 5, 6},
                    {7, 8, 9} };
    printf("%d\n", Sum(&a, 3, 3));
    return 0;
}
```

48、二维数组与二维数组指针

题目：有如下定义

```
int a[3][4] = { {1, 2, 3, 4},
                {5, 6, 7, 8},
                {9, 10, 11, 12} };
```

```
int (*p)[3][4] = &a;    //二维数组指针
```

以下选项，分别指代的是什么

A: $*((\text{int}*) (p+1)-2) = 3$

B: $*((\text{int}*) ((\text{int}*) [4])p+2) - 2) = 3$

C: $*(\text{int}*) ((\text{int}*) [2])p+3) = 4$

解析：

1、二维数组结构图



2、知识点，数组涉及指针操作三定理

数组名即首元素首地址

指针加减 1，加减的值为指针类型的字节数大小

$*(p+1) == p[1]$

选项：

A: $*((\text{int}*) (p+1)-2) = 3$

1、p 是二维数组类型指针，p+1 加整个二维数组的大小，指向下一个二维数组的首地址，即 12 元素的相邻

2、(int*)将该地址强转给 int*

3、-2，由于地址强转为元素类型，所以-2 为减两个元素大小，指向 11 元素的地址，&a[2][1]

4、*&a[2][1]即为 a[2][1]

5、a[2][1] = 3

B: $*((\text{int}^*)((\text{int}^*)[4])p+2) - 2 = 3$

- 1、 $(\text{int}^*)[4]p$, 将 p 转为一维数组类型指针
- 2、+2, 即加 2 个小数组大小, 为 $\&a[2]$
- 3、 $(\text{int}^*)\&a[2]$, 转换为 int^* , 等同于 $\&a[2][0]$
- 4、-2, 为 $\&a[2][0]-2$ 是减两个元素大小, 为 $\&a[1][2]$
- 5、 $*\&a[1][2]$ 即 $a[1][2]$
- 6、 $a[1][2] = 3$

C: $*(\text{int}^*)((\text{int}^*)[2])p+3) = 4$

- 1、 $(\text{int}^*)[2]p$, 将 p 转为 2 元素的数组类型地址
- 2、+3, 加 3 个数组大小, 是 6 个元素, 即 7 8 元素构成的数组地址
- 3、 (int^*) 将地址转换成元素类型, 此时为 7 元素的地址, $\&a[1][2]$
- 4、 $*\&a[1][2]$ 为 $a[1][2]$
- 5、 $a[1][2] = 4$

49、矩阵转置

题目：编程实现如下矩阵转置，函数头已给出

```
#define hang 3
#define lie 2
int arr[hang][lie] = { {1,2}, {3,4}, {5,6} };
void ZhuanZhi(int arr[hang][lie], int crr[lie][hang])
{
}
```

分析：

1、矩阵转置

1	2				
3	4	1	3	5	
5	6	2	4	6	

转置前的第 1 行，转置后为第 1 列

转置前的第 2 行，转置后为第 2 列

转置前的第 3 行，转置后为第 3 列

2、下标位置分析

1	2	3	4	5	6
a[0][0]	a[0][1]	a[1][0]	a[1][1]	a[2][0]	a[2][1]

转置后

1	2	3	4	5	6
a[0][0]	a[1][0]	a[0][1]	a[1][1]	a[0][2]	a[1][2]

发现横纵下标刚好是位置互换

$crr[i][j] = arr[j][i];$

完整代码

```
#include <stdio.h>
```

```
#define hang 3
```

```
#define lie 2
```

```
void ZhuanZhi(int arr[hang][lie], int crr[lie][hang])
{
    int i, j;
```

```
    for (i = 0; i < lie; i++)
    {
        for (j = 0; j < hang; j++)
        {
            crr[i][j] = arr[j][i];
        }
    }
}

int main(void)
{
    int arr[hang][lie] = { {1,2}, {3,4}, {5,6} };
    int crr[lie][hang] = { 0 };

    return 0;
}
```